Practical : 10

Name: Payal Ganesh Ghanwat Roll No. : 222 Class: SE

Problem Statement: AI search Algorithm -CO1, CO2, CO4, CO5 To design and implement a maze navigation system that enables an agent to find an optimal path from a starting point to a goal using AI search algorithms. The system should be capable of solving both static and dynamic mazes and should visualize the pathfinding process.

Code:

```
import heapq

# Maze Representation
# 0 = Free Path
# 1 = Wall

maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0]
]

ROWS = len(maze)
COLS = len(maze[0])

start = (0, 0)
goal = (4, 4)


# Heuristic Function (Manhattan Distance)
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])


# A* Search Algorithm def
astar(maze, start, goal):

    priority_queue = []

    heapq.heappush(priority_queue, (0, start))
```

```
    came_from = {} g_cost = {start: 0} while

    priority_queue: current =

    heapq.heappop(priority_queue)[1] if current ==

    goal: path = []

            while current in came_from:

        path.append(current) current =

        came_from[current]

        path.append(start) path.reverse()

        return path x, y = current

        directions = [(0,1),(1,0),(0,-1),(-1,0)]

        for dx, dy in directions:

            nx = x + dx ny = y + dy if 0 <= nx <

            ROWS and 0 <= ny < COLS:

                if maze[nx][ny] == 0: new_cost = g_cost[current] + 1

                    neighbor = (nx, ny) if neighbor not in g_cost or new_cost

                    < g_cost[neighbor]:

                        g_cost[neighbor] = new_cost f_cost =

                        new_cost + heuristic(neighbor, goal)

                        heapq.heappush(priority_queue, (f_cost,

                        neighbor)) came_from[neighbor] = current

                        return None


# Display Maze def
printMaze(maze, path):
```

```
display = []

for row in maze:
    display.append(row[:])

if path:

    for x, y in path:

        display[x][y] = "*"

    sx, sy = start
    gx, gy = goal

    display[sx][sy] = "S"

display[gx][gy] = "G"

print("\nSolved Maze\n")

for row in display:

    for cell in row:

        if cell == 1:
            print("", end=" ")

        elif cell == "*":
            print("*", end=" ")

        elif cell == "S":
            print("S", end=" ")

        elif cell == "G":

        print("G", end=" ")


        else:
            print(".", end=" ")

    print()
```

```
# Main Program path = astar(maze,

start, goal) if path: print("\nShortest

Path Found\n") print(path)

print("\nPath Length =", len(path)-1)

printMaze(maze, path) else: print("No

Path Found")
```

Output:
Shortest Path Found

[(0,0),
(0,1),
(0,2),
(1,2),
(2,2),
(2,3),
(2,4),
(3,4),
(4,4)]

Path Length = 8

Solved Maze

S * * ▉ .

▉▉ * ▉ .
. . * * *
. ▉▉▉ *
. . . ▉ G

OR

Code: import

heapq

maze = [

```
    [0,0,0],
    [1,1,0],
    [0,0,0]
]

start = (0,0)
goal = (2,2)

def heuristic(a,b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

pq = [(0, start)]
visited = set()
parent = {}

while pq: cost, current =
    heapq.heappop(pq)

    if current == goal:
        break

    if current in visited:

    continue

    visited.add(current)

    x, y = current

    for dx, dy in [(0,1),(1,0),(0,-1),(-1,0)]: nx, ny = x+dx,

        y+dy if 0 <= nx < 3 and 0 <= ny < 3 and maze[nx][ny]

        == 0:

            if (nx, ny) not in visited: heapq.heappush(pq, (cost + 1 +
                heuristic((nx,ny), goal), (nx,ny))) parent[(nx,ny)] = current

# Find Path
path = []
node = goal

while node != start:
    path.append(node)
    node = parent[node]
```

```
path.append(start)

path.reverse()

print("Shortest Path:", path)

# -------- Visualization -------display
= []

for row in maze:
    display.append(row[:])

for x, y in path:
    display[x][y] = "*"

display[start[0]][start[1]] = "S"

display[goal[0]][goal[1]] = "G"

print("\nMaze Visualization:\n")

for row in display:
    for cell in row:
        if cell == 1: print("#",
            end=" ")
        elif cell == "*":
            print("*", end=" ")
        elif cell == "S":
            print("S", end=" ")
        elif cell == "G":
            print("G", end=" ")
        else:
            print(".", end=" ")
    print()
```

Output:

Shortest Path: [(0, 0), (0, 1), (0,

2), (1, 2), (2, 2)] Maze

Visualization:

```
S * *
# # *
. . G
```